# Desafío II: Fiesta mundial con UdeAWorldCup


1st jesús Sebastián Mendoza
*dept. Ingenieria Electrónica*
*Universidad de Antioquia*
Medellín, Antioquia
jsebastian.mendoza@udea.edu.co

2nd Danna Isabela ochoa
*dept. Ingenieria Electrónica*
*Universidad de Antioquia*
Medellín, Antioquia
danna.ochoa@udea.edu.co


## I. Análisis del problema y consideraciones

El sistema UdeAWorldCup modela la Copa Mundial FIFA 2026 con 48 equipos, 26 jugadores por equipo y 104 partidos distribuidos en seis etapas. El problema tiene tres ejes principales: gestionar datos de equipos y jugadores, simular partidos con resultados probabilísticos coherentes, y producir estadísticas finales del torneo.

La primera decisión de diseño fue la separación estricta de responsabilidades. Cada clase tiene una razón de existir delimitada: EstadisticasJugador y EstadisticasEquipo solo guardan y acumulan números, Partido contiene toda la lógica de simulación, Grupo organiza la fase de grupos, EtapaEliminatoria gestiona el fixture eliminatorio, Torneo coordina el flujo general, Vista presenta resultados en consola y GestionarArchivosCSV maneja exclusivamente la entrada y salida de archivos.

La segunda decisión fue la prohibición de STL. El enunciado prohíbe usar contenedores estándar, lo que obligó a implementar ArregloDinamico¡T¿, una plantilla genérica propia que reemplaza a std::vector. Esto implica gestionar manualmente la memoria con new y delete en cada clase que almacena colecciones.

La tercera decisión fue la regla de los tres en todas las clases con punteros. Jugador tiene char* para nombre y apellido. Equipo tiene cuatro char* y una plantilla de 26 jugadores. Partido tiene char* para fecha, hora, sede y árbitros. Cada una de estas clases implementa constructor de copia, operador de asignación y destructor para garantizar copias profundas y evitar memory leaks o punteros colgantes.

La cuarta decisión fue implementar las matemáticas sin ¡cmath¿. La ecuación de goles esperados requiere exponentes decimales (0.6 y 0.4) que normalmente se calcularían con pow(). Sin esa función, se convirtieron los exponentes a fracciones (3/5 y 2/5) y se implementó una aproximación de raíz quinta usando el método de Newton-Raphson en 15 iteraciones.

La quinta decisión fue distribuir el contador de iteraciones dentro de cada clase. Cada módulo cuenta sus propias operaciones y Torneo los agrega en getIteraciones(), permitiendo mostrar métricas reales sin necesitar un objeto externo de medición..


Identify applicable funding agency here. If none, delete this.


## II. Diagrama de clases

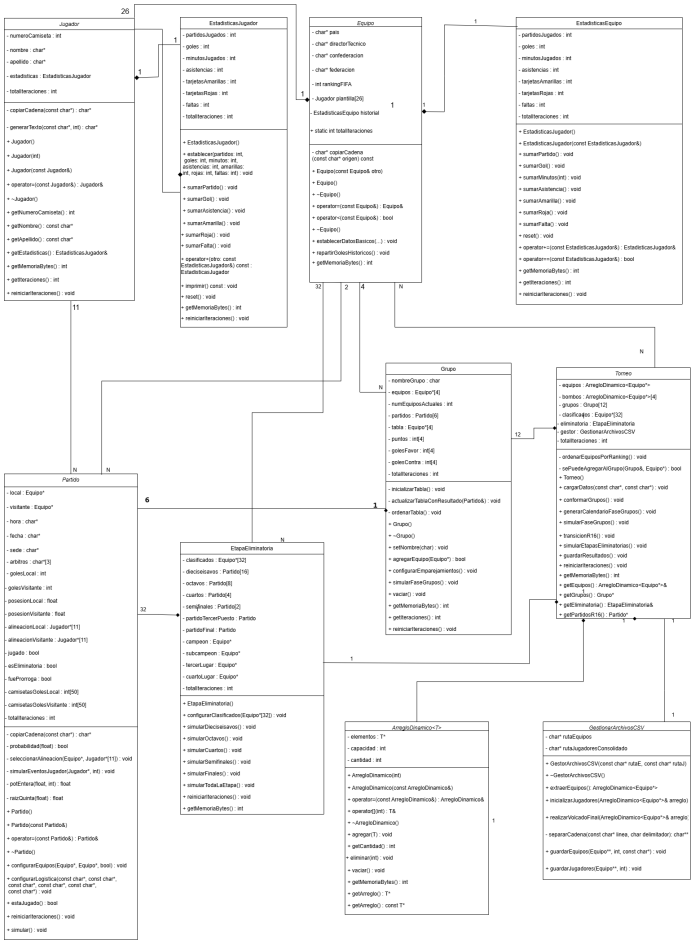


Fig. 1. Diagrama de clases.

## III. Algoritmos esenciales

### A. Prórroga en partidos eliminatorios

Si un partido eliminatorio termina empatado, se usa el porcentaje de posesión del equipo local como probabilidad

de sesgo. Se genera un número aleatorio y si cae dentro del porcentaje de posesión local, gana el local; de lo contrario gana el visitante. Se suma un gol al marcador del equipo ganador, se registra el número de camiseta del jugador aleatorio que lo anotó, y los minutos de todos los jugadores del partido se actualizan a 120.

### *B. Lectura y escritura de datos desde archivos CSV*

El sistema implementa un algoritmo propio para la lectura y escritura de datos en archivos, debido a la restricción de no utilizar librerías avanzadas ni estructuras de alto nivel. Este proceso es gestionado por la clase encargada de manipulación de archivos.

*1) Lectura de datos:* La lectura de datos se realiza desde un archivo CSV delimitado por punto y coma. El algoritmo sigue un enfoque secuencial carácter por carácter. El proceso general es el siguiente:

1) Se abre el archivo en modo lectura y se descartan las primeras líneas correspondientes a encabezados.
2) Cada línea del archivo se procesa de forma independiente mediante un cursor que recorre el arreglo de caracteres.
3) Para cada campo:
   - Se identifica el inicio del dato.
   - Se avanza hasta encontrar el delimitador (`;`) o el final de línea.
   - Se reemplaza temporalmente el delimitador por un carácter nulo para aislar el campo.
   - Se crea una copia dinámica del texto.
4) Los valores numéricos se convierten manualmente mediante acumulación de dígitos.
5) Con los datos obtenidos se instancia un nuevo objeto de tipo `Equipo`, asignando sus atributos básicos y estadísticas.
6) Finalmente, el objeto se inserta en una estructura dinámica de almacenamiento.

*2) Escritura de datos:* La escritura de datos se utiliza para exportar las estadísticas de los jugadores al finalizar la simulación del torneo. El algoritmo genera un archivo por cada equipo.

El proceso consiste en:

1) Recorrer la colección de equipos almacenados en memoria.
2) Para cada equipo:
   - Se construye dinámicamente el nombre del archivo concatenando el nombre del país con un sufijo fijo.
   - Se abre un archivo de salida en modo escritura.
3) Se escribe una línea de encabezado con los campos correspondientes.
4) Se recorre la plantilla de jugadores del equipo.
5) Para cada jugador:
   - Se accede a sus estadísticas individuales.
   - Se escriben los datos en formato tabular, separados por tabuladores.
6) Se cierra el archivo y se libera la memoria utilizada para el nombre del archivo.

### *C. Sorteo y conformación de grupos*

El proceso de sorteo distribuye a las 48 selecciones clasificadas en 12 grupos (A-L) garantizando que se respeten las restricciones geográficas reales de la FIFA. El algoritmo opera mediante un enfoque probabilístico con retroceso (*backtracking* mitigado), operando de la siguiente manera:

1) Se ordenan los equipos en orden descendente según su Ranking FIFA para distribuirlos de forma equitativa en cuatro bombos de 12 equipos cada uno. Excepcionalmente, el país anfitrión (EE. UU.) se asigna por defecto al Bombo 1.
2) Se itera sobre cada bombo y cada grupo intentando asignar un equipo seleccionado de forma pseudoaleatoria.
3) Antes de confirmar la inserción en el grupo, el sistema evalúa una regla estricta de confederaciones: no puede haber dos equipos de la misma confederación en un mismo grupo, con la excepción de la UEFA, que permite un máximo de dos representantes.
4) Dado que la asignación completamente aleatoria puede desencadenar un punto muerto (*deadlock*) donde los equipos restantes en los bombos no cumplen con los criterios para los grupos disponibles, el algoritmo implementa un límite de 500 intentos por asignación. Si se supera este límite, se determina un fallo en el árbol de decisiones, vaciando los grupos y reiniciando el sorteo desde cero hasta encontrar una distribución matemáticamente válida.

### *D. Ordenamiento y criterios de desempate*

Debido a las restricciones del proyecto que prohíben el uso de funciones de ordenamiento de la biblioteca estándar (como `std::sort`), se requiere una implementación manual estricta. El sistema hace uso del algoritmo de ordenamiento de la burbuja (*Bubble Sort*), el cual fue adaptado para manejar múltiples criterios de evaluación de forma secuencial:

- **Fase de Grupos:** Tras simular los partidos, la tabla de posiciones interna de cada grupo se ordena evaluando como primer criterio los puntos obtenidos. En caso de igualdad, el algoritmo desciende al segundo criterio (diferencia de goles) y, si la igualdad persiste, al tercer criterio (mayor cantidad de goles a favor).
- **Clasificación de Mejores Terceros:** Para determinar los 8 equipos que avanzan a los dieciseisavos de final, el algoritmo extrae a los terceros lugares de los 12 grupos y traslada sus estadísticas a arreglos paralelos. Durante el ordenamiento de la burbuja, si las validaciones de desempate indican que debe haber un cambio de posición, el algoritmo intercambia de manera síncrona y simultánea los punteros de los equipos, los puntos, la diferencia de goles y los goles a favor para mantener la integridad relacional de los datos.

### E. Recolección descentralizada de métricas de rendimiento

Para evaluar la complejidad algorítmica y el uso de recursos computacionales sin alterar el flujo del simulador, el sistema implementa un algoritmo de recolección de métricas espaciales (memoria) y temporales (iteraciones) fuertemente fundamentado en los principios de la Programación Orientada a Objetos:

- **Complejidad Temporal (Iteraciones):** En lugar de depender de variables globales, el sistema utiliza contadores locales de instancia. Cada objeto (como el gestor de archivos, los grupos o las fases eliminatorias) es responsable de contabilizar los ciclos de reloj de sus propios bucles internos. Al finalizar un macroproceso, la clase orquestadora principal (`Torneo`) consulta, recolecta y acumula estos valores parciales.
- **Complejidad Espacial (Memoria en bytes):** El cálculo de la huella de memoria se realiza mediante un algoritmo de recolección en cascada. Cada módulo de software incluye un método que calcula su propio peso estático (`sizeof(*this)`) y le suma de forma iterativa la memoria dinámica generada internamente por sus arreglos o listas enlazadas subyacentes, reportando finalmente el valor total al componente padre.

## IV. Problemas de desarrollo y respectivas solución

### A. Confederaciones repetidas en la conformación de grupos

Uno de los problemas más complejos durante el desarrollo fue garantizar que los grupos cumplieran correctamente las restricciones de confederación establecidas por el enunciado. Específicamente, se debía evitar que un grupo tuviera más de un equipo de la misma confederación, con la excepción de UEFA, que permite máximo dos representantes.

En una primera versión del algoritmo, únicamente se validaba que no hubiera más de dos equipos pertenecientes a UEFA dentro de un mismo grupo. Sin embargo, no se controlaba de manera estricta la repetición de otras confederaciones, lo que generaba configuraciones inválidas.

Posteriormente, se intentó corregir este comportamiento introduciendo un mecanismo basado en intentos. El algoritmo realizaba múltiples selecciones aleatorias de equipos (hasta aproximadamente 100 iteraciones) buscando una combinación válida que cumpliera las restricciones. No obstante, este enfoque resultó insuficiente, ya que en ciertas configuraciones el sistema quedaba bloqueado al no encontrar una solución válida dentro de los intentos permitidos.

Para solucionar este problema, se diseñó una estrategia más robusta basada en validación estricta y reinicio del proceso. Se implementó una función encargada de verificar si un equipo puede ser agregado a un grupo sin violar las restricciones. Esta función recorre los equipos ya asignados y compara sus confederaciones con la del candidato.

El procedimiento funciona de la siguiente manera:

1) Se obtiene la confederación del equipo candidato.
2) Se recorren los equipos ya presentes en el grupo.
3) Si el candidato pertenece a UEFA, se cuenta cuántos equipos UEFA existen:
   - Si el conteo alcanza dos, se rechaza la inserción.
4) Si el candidato pertenece a otra confederación:
   - Se verifica que no exista ya un equipo de esa misma confederación.
   - En caso de existir, se rechaza la inserción.
5) Si ninguna restricción se viola, el equipo es válido.

Aun con esta validación, surgía un segundo problema: en ciertas distribuciones de los bombos, podía ocurrir que ningún equipo restante fuera válido para un grupo específico. Esto impedía completar la asignación.

Para resolver esta situación, se implementó un mecanismo de recuperación global. Si después de múltiples intentos no se logra asignar un equipo válido a un grupo, el algoritmo vacía completamente los grupos y los bombos, reiniciando el proceso de sorteo desde cero. Este procedimiento se repite hasta encontrar una configuración válida.

Este enfoque garantiza que todas las restricciones se cumplan finalmente, aunque implica un mayor costo computacional en casos extremos. Sin embargo, permite manejar adecuadamente la combinación de aleatoriedad y restricciones del problema.

En conclusión, la solución evolucionó desde una validación parcial basada en intentos, hacia un algoritmo más robusto que combina validación estricta con reinicio controlado, asegurando el cumplimiento de todas las condiciones del sorteo.

### B. Algoritmo de asignación de fechas con máximo de partidos diarios y descanso mínimo

El calendario de la fase de grupos debe cumplir dos restricciones principales: no programar más de cuatro partidos en un mismo día y garantizar que un equipo no juegue nuevamente hasta haber descansado al menos tres días.

Para resolver esto, el algoritmo asigna las fechas de manera secuencial, evaluando cada partido antes de fijarlo definitivamente en el calendario. Para cada encuentro se conserva un contador de partidos por día y un registro de la última fecha en la que jugó cada equipo.

*1) Control del máximo de cuatro partidos por día:* El sistema mantiene una variable que indica cuántos partidos han sido programados en el día actual. Cada vez que se asigna un partido a una fecha, este contador aumenta en uno. Si el contador llega a cuatro, el algoritmo avanza al siguiente día del torneo y reinicia el contador en cero.

De esta forma, la lógica garantiza que ningún día tenga más de cuatro partidos. El proceso puede describirse así:

1) Se inicia el calendario desde el día 20 de junio de 2026.
2) Se inicializa el contador de partidos del día en cero.
3) Cada vez que se programa un partido, se incrementa el contador.
4) Si el contador alcanza el valor de cuatro, se avanza al día siguiente.
5) El contador vuelve a cero para comenzar la programación del nuevo día.

Esta restricción permite distribuir los partidos de forma ordenada y evita la acumulación excesiva de encuentros en una misma fecha.

*2) Verificación del descanso mínimo de tres días:* Para controlar que un equipo tenga al menos tres días de separación entre partidos, el algoritmo debe registrar la última fecha en la que jugó cada selección. Antes de programar un nuevo partido, se consulta la última fecha registrada para el equipo local y para el equipo visitante.

Si alguno de los dos equipos jugó hace menos de tres días, el partido no puede asignarse a la fecha actual. En ese caso, el algoritmo avanza al siguiente día y vuelve a evaluar la condición hasta encontrar una fecha válida.

El procedimiento general es:

1) Para cada equipo se almacena el último día en que jugó.
2) Antes de programar un partido, se calcula la diferencia entre el día actual y el último día jugado por cada equipo.
3) Si ambos equipos cumplen el descanso mínimo, el partido se agenda en esa fecha.
4) Si alguno no cumple, se avanza al siguiente día y se repite la verificación.
5) Una vez asignado el partido, se actualiza la última fecha jugada por ambos equipos.

La condición principal puede expresarse de forma lógica como:

$$diaActual - ultimoDiaEquipo \geq 3$$

Esto significa que el equipo solo puede jugar si han pasado al menos tres días desde su último partido.

*3) Funcionamiento combinado:* Ambas restricciones deben evaluarse al mismo tiempo. Por lo tanto, un partido solo puede ser programado si se cumplen dos condiciones:

- El día actual tiene menos de cuatro partidos asignados.
- Tanto el equipo local como el visitante han descansado al menos tres días.

Si una de estas condiciones falla, el algoritmo no agenda el partido en esa fecha y prueba con el día siguiente. Cuando ambas condiciones se cumplen, se asigna la logística del partido.

### C. Dificultades en la lectura del archivo

Uno de los primeros problemas abordados en el desarrollo fue la lectura de datos desde archivos CSV. Debido a la restricción de no utilizar librerías de alto nivel como STL, no fue posible emplear herramientas estándar como `stringstream` o funciones avanzadas de procesamiento de cadenas.

Inicialmente, la lectura de datos presentaba inconsistencias debido a la dificultad de separar correctamente los campos delimitados por punto y coma. Esto llevó a errores en la asignación de atributos, especialmente en valores numéricos y cadenas de caracteres.

Para resolver este problema, se implementó un algoritmo de lectura manual carácter por carácter, en el cual cada línea del archivo se recorre utilizando un puntero. Al encontrar un delimitador, este se reemplaza temporalmente por un carácter nulo, permitiendo aislar el campo actual. Posteriormente, se realiza una copia dinámica del contenido para almacenarlo en memoria.

Adicionalmente, los valores numéricos se convierten manualmente mediante acumulación de dígitos, evitando el uso de funciones estándar. Este enfoque, aunque más complejo, permitió tener control total sobre la lectura de datos y garantizar la correcta inicialización de los objetos `Equipo`.

### D. Selección de una estructura de datos eficiente y accesible

Otro reto importante fue definir una estructura de datos que permitiera almacenar y manipular la información de manera eficiente, cumpliendo al mismo tiempo con las restricciones del enunciado.

La ausencia de STL implicó la necesidad de diseñar una estructura propia. En una primera aproximación se consideró el uso de arreglos estáticos, sin embargo, esta opción resultaba limitada debido a la necesidad de manejar colecciones dinámicas de equipos.

Como solución, se implementó la plantilla `ArregloDinamico<T>`, la cual permite almacenar elementos de forma secuencial con acceso por índice, inserción y eliminación. Esta estructura fue utilizada en la clase `Torneo` para gestionar los equipos cargados desde el archivo.

El principal desafío fue equilibrar eficiencia y simplicidad. Se buscó que la estructura fuera lo suficientemente flexible para soportar las operaciones necesarias, pero sin introducir una complejidad excesiva en la gestión de memoria. Esto permitió mantener un acceso directo a los elementos, lo cual es fundamental en algoritmos como ordenamiento por burbuja y selección de clasificados.

### E. Sincronización de la ejecución entre clases

Durante el diseño del sistema, uno de los problemas más relevantes fue garantizar que cada componente se ejecutara en el momento adecuado dentro del flujo del programa.

Dado que el sistema está compuesto por múltiples clases interdependientes (como Torneo, Grupo, Partido y EtapaEliminatoria), era necesario definir un orden de ejecución claro para evitar inconsistencias en los datos.

En versiones iniciales, algunos métodos eran llamados fuera de contexto, lo que generaba problemas como simulaciones sin datos cargados o acceso a estructuras no inicializadas. Esto evidenció la necesidad de centralizar el control del flujo.

La solución fue concentrar la lógica principal en la clase `Torneo`, la cual actúa como coordinador del sistema. En esta clase se define una secuencia clara de ejecución: primero se cargan los datos, luego se conforman los grupos, posteriormente se genera el calendario, se simula la fase de grupos, se realiza la transición a eliminatorias y finalmente se ejecuta la fase eliminatoria.

Este enfoque permitió asegurar que cada módulo se ejecutara en el momento correcto, garantizando coherencia en los datos y evitando errores de dependencia.

### F. Cálculo de la memoria utilizada

El cálculo de memoria representó otro desafío significativo, ya que el sistema debía reportar la cantidad exacta de memoria utilizada sin depender de herramientas externas.

El problema principal radicaba en que muchas clases utilizan memoria dinámica, especialmente aquellas que manejan cadenas de caracteres mediante punteros char. Esto implica que el uso de `sizeof()` por sí solo no es suficiente, ya que este operador solo considera el tamaño de los punteros y no el contenido al que apuntan.

Para resolver esto, se implementó un método específico en cada clase encargado de calcular su consumo de memoria. Este método suma el tamaño base del objeto y, adicionalmente, recorre cada atributo dinámico para calcular la memoria real utilizada.

En la clase Torneo, este proceso se extiende agregando la memoria utilizada por los grupos, la etapa eliminatoria y cada uno de los equipos almacenados. Este enfoque permite obtener una estimación precisa del uso de memoria del sistema completo.

Aunque este cálculo incrementa la complejidad del código, proporciona una visión detallada del consumo de recursos, lo cual es especialmente relevante en un entorno donde se trabaja con memoria dinámica de forma manual.

## V. Conclusiones

El desarrollo del sistema UdeAWorldCup demostró que la imposición de restricciones técnicas rigurosas —como la prohibición de la biblioteca estándar (STL) y funciones matemáticas avanzadas— no solo es un desafío de codificación, sino un catalizador fundamental para la comprensión profunda de las estructuras de datos computacionales y el manejo de memoria a bajo nivel. La implementación desde cero de plantillas genéricas como `ArregloDinamico<T>` y la resolución matemática mediante métodos numéricos (Newton-Raphson) validaron la viabilidad de construir software complejo utilizando únicamente los fundamentos primitivos del lenguaje C++.

Desde una perspectiva arquitectónica, el proyecto evidenció que el éxito de una simulación a gran escala radica en la separación estricta de responsabilidades y el diseño orientado a objetos. Al delegar la orquestación del flujo a la clase `Torneo` y descentralizar las tareas específicas (como la simulación de probabilidades en `Partido` o la serialización en `GestionarArchivosCSV`), se logró un acoplamiento bajo y una alta cohesión. Esto previno condiciones de carrera lógicas y aseguró la sincronización en un sistema donde las dependencias de datos son estrictamente secuenciales.

En el ámbito algorítmico, la resolución de restricciones del mundo real (como las reglas de confederaciones de la FIFA y los descansos mínimos de tres días en el calendario) requirió ir más allá de la fuerza bruta. La implementación de un modelo de retroceso con reinicio global (*backtracking*) para el sorteo, junto con la evaluación secuencial de estados para la programación de fechas, demostró que la combinación de heurísticas y validaciones de estado es esencial para evitar bloqueos lógicos (*deadlocks*) en problemas de asignación múltiple.

Finalmente, el diseño de la recolección de métricas confirmó que la medición del rendimiento computacional (complejidad temporal y espacial) no tiene por qué romper la encapsulación del software. El uso de contadores de instancia y el cálculo dinámico de memoria en cascada —sumado a la rigurosa aplicación de la "Regla de los Tres" para evitar fugas de memoria (*memory leaks*)— dio como resultado un sistema que no solo modela fielmente un evento deportivo masivo, sino que también es capaz de auditar su propio rendimiento y huella de memoria de manera autónoma, robusta y escalable.